

Từ Diffusion đến Flow:

Tư duy lại về các mô hình sinh ảnh

by Minh Huu Nguyen



TABLE OF CONTENT

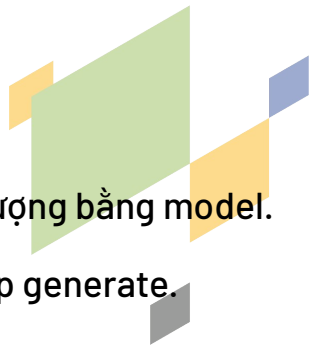
1. Nhóm các mô hình Image Generation
2. Flow matching là gì?
3. Flow matching trong FLUX/FLUX-Kontext



1. Nhóm các mô hình Image Generation

Decorative geometric shapes on the right side of the slide, including a large green parallelogram, a yellow parallelogram, and a small black parallelogram.

Các trục phân loại



1. Likelihood explicit vs implicit

- a. **Likelihood explicit:** Mô hình cho ra một hàm mật độ, có thể tính trực tiếp hoặc ước lượng bằng model.
- b. **Likelihood implicit:** Mô hình ko cho ra một hàm mật độ mà chỉ cung cấp phương pháp generate.

2. Deterministic vs stochastic sampling

- a. **Deterministic sampling:** từ latent thành ảnh bằng một quy tắc xác định.
- b. **Stochastic sampling:** từ latent thành ảnh có các thành phần ngẫu nhiên.

3. Invertible vs non-invertible

- a. **Invertible:** có thể biến đổi hai chiều giữa latent và ảnh.
- b. **Non-invertible:** ánh xạ từ latent thành ảnh không khả nghịch hoặc không có ánh xạ từ latent thành ảnh.

Một số họ mô hình



Họ mô hình	Likelihood explicit / implicit	Deterministic / stochastic sampling	Invertible / non-invertible
Flow-based model: - Normalizing Flows (NF) - Continuous Normalizing Flows (CNF)	explicit	deterministic	invertible
Flow-based model: - Flow matching (Rectified Flow (RF))	implicit	deterministic	non-invertible
Variational Autoencoder (VAE)	explicit	stochastic (sampling latent) deterministic (decoder)	non-invertible
Generative Adversarial Networks (GANs)	implicit	deterministic	non-invertible
Diffusion/score-based models	implicit	stochastic	non-invertible

2. Flow matching là gì?



Ý tưởng chính của Flow matching

Huấn luyện mô hình học một trường **vận tốc** sao cho **tích phân đường của vector đó** (là đường đi) chuyển một **phân phối cơ sở** (ví dụ Gaussian) thành **phân phối dữ liệu**.

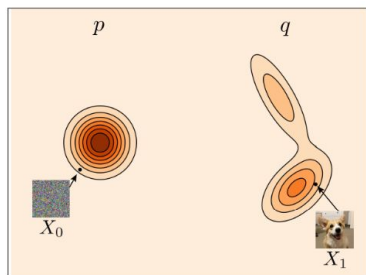
Nói nôm na...

Mục tiêu của flow matching xây **một con đường** từ noise thành ảnh.

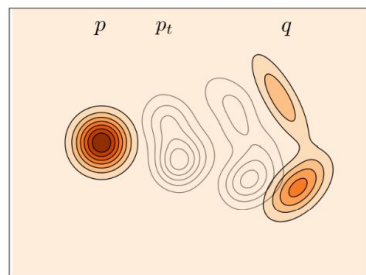
Mô hình được huấn luyện theo flow matching sẽ biết được khi nó **xuất phát ở đâu đường là noise hoặc một ví trị nào đó ở trên đường** thì nó phải **đi như thế nào** (hướng + độ lớn) để đi được đến **đích là ảnh**.



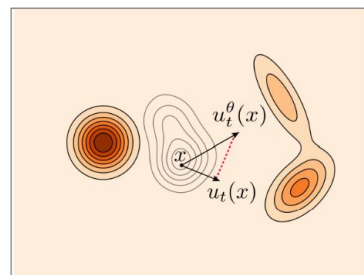
Các bước thực hiện



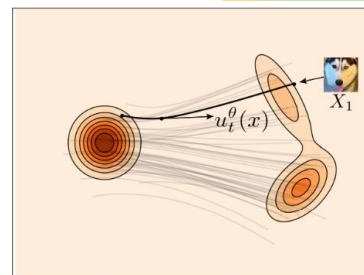
(a) Data.



(b) Path design.



(c) Training.



(d) Sampling.

- Hình a, ta có **noise p** và **phân phối dữ liệu thật q**.
- Hình b, ta định nghĩa một đường đi từ **p** đến **q** sử dụng hàm nội suy, từ đó tạo ra các **pt** trung gian tại các vị trí **t**.
- Hình c, ta huấn luyện mô hình:
 - Đầu vào: **noise x** tại một trong các **pt**.
 - Đầu ra: **vận tốc** (hướng + độ lớn) tại vị trí **x** để đi được đến **q**.
- Hình d, trong inference, với **infer_step=k** được chọn trước, ta sample **x** tại **p** (có thể gọi là **p0**), lần lượt tính **p1, p2, ..., pt, ..., pk**.

So sánh Flow matching với Diffusion model

```
def interpolate(x0, x1, t):
    return (1 - t.view(-1,1,1,1)) * x0 + t.view(-1,1,1,1) * x1

for epoch in range(epochs):
    # x1: real image batch
    for x1 in dataloader:
        # t: sample t in [0,1]
        t = torch.rand(x1.shape[0])
        # x0: base noise sample
        x0 = torch.randn_like(x1)
        # xt: position along path
        xt = interpolate(x0, x1, t)
        # for linear interpolation, dx/dt = x1 - x0
        velocity_true = (x1 - x0)

        # model predicts the velocity at (xt, t)
        v_pred = model(xt, t)
        loss = mse(v_pred, velocity_true)
        opt.zero_grad()
        loss.backward()
        opt.step()
```

Flow
Matching

```
def q_sample(x0, eps, t):
    return x0 * (1 - t) + eps * t

for epoch in range(epochs):
    # x0: real image batch
    for x0 in dataloader:
        # t: sample t in [0,1]
        t = torch.rand(x0.shape[0])
        # eps: noise sample
        eps = torch.randn_like(x0)
        # xt: noisy image at timestep t
        xt = q_sample(x0, eps, t)

        # model predicts the added noise
        eps_pred = model(xt, t)
        loss = mse(eps_pred, eps)
        opt.zero_grad()
        loss.backward()
        opt.step()
```

Diffusion
Model

3. Flow matching trong FLUX/FLUX-Kontext

Decorative geometric shapes on the right side of the slide, including a large green parallelogram, a yellow parallelogram, and a small black parallelogram.

Hàm loss flow matching trong FLUX/FLUX-Kontext

$$\mathcal{L}_\theta = \mathbb{E}_{t \sim p(t), x, y, c} \left[\|v_\theta(z_t, t, y, c) - (\varepsilon - x)\|_2^2 \right],$$

- x là ảnh thật (ảnh target) để training.
- y là ảnh context (ảnh đầu vào điều kiện), optional.
- c là text prompt.
- t là "vị trí" ở giữa $[0, 1]$ (giữa [noise, ảnh thật]) $\rightarrow$ qua hàm interpolation $\rightarrow$ tạo ra z_t latent noise "trung gian".
- v_θ là model cần train với tham số φ .
- ε là noise ở điểm xuất phát $\rightarrow (\varepsilon - x)$ là target để tính loss.
- $E \dots$ là MSE loss.

Giải thích vì sao $(\varepsilon - x)$ là target để tính loss?

For training our models, we construct forward noising processes in the latent space of an image autoencoder as

$$z_t = a_t x_0 + b_t \varepsilon, \quad (4)$$

with $x_0 \sim p_{data}$, $\varepsilon \sim \mathcal{N}(0, 1)$, and the coefficients a_t and b_t define the log signal-to-noise ratio (log-SNR) [25]

$$\lambda_t = \log \frac{a_t^2}{b_t^2} \quad (5)$$

Further, we use the conditional flow matching loss [30]

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t \sim p(t), \varepsilon \sim \mathcal{N}(0,1)} \left\| v_{\Theta}(z_t, t) - \frac{a'_t}{a_t} z_t + \frac{b_t}{2} \lambda'_t \varepsilon \right\|_2^2 \quad (6)$$

For rectified flow models [32], $a_t = 1 - t$ and $b_t = t$, and thus

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t \sim p(t), \varepsilon \sim \mathcal{N}(0,1), x_0 \sim p_{data}} \left\| v_{\Theta}(z_t, t) + x_0 - \varepsilon \right\|_2^2 \quad (7)$$

